The best keyboard layout for a Vim programmer

H j k l, Esc, and how much Vim matters

Joshua Markle

University of Virginia · October 2026

Experiment report · 25 October 2026 · GECKO commit baad493 + this study · corne-36 · `vim-programmer-typed` corpus (prose 35, Markdown 8, code as typed 35, Vim sessions 22, Backspace 5.6 %) · profiles `vim-*` (the programming study's `prog-cal` plus an h j k l policy). Data and scripts: `bench/vim-layout-2026-10-25/`.

Abstract

A programmer who uses Vim types text and commands: motions, ⎋, `:` and `/`. We search corne-36 for the best layout for that mix (22 % synthetic Vim sessions, the rest prose, Markdown and code as typed), with the measured layer-press cost from the programming study, under five ways of keeping h j k l usable. **Keeping h j k l costs almost nothing.** With enough search, every policy reaches the same score within 0.1 %: h–l and j–k as neighbour pairs (+0.05 %), h j k l in a row (+0.1 %), on four fingers of one hand (+0.1 %); even pinning them to their QWERTY keys costs only 0.7 % (0.05 % of typing time). Shorter searches suggested 1–3 % gaps that were search noise. **Esc belongs on a base thumb.** Every optimum with Vim in the text puts ⎋ on the left outer thumb and moves Enter onto the symbol layer; `:` and `/` go to the symbol layer's home row. The programming optimum with Esc on a layer is 4.7 % worse on the Vim mix, while the Vim optimum gives up only 0.4 % on programming without Vim. **One layout serves 11–33 % Vim:** the 22 % optimum is the best found on every share. The limits: the Vim keystrokes are synthetic (no population data exist), the layer cost comes from Shift, and the gaps between policies are far below what a typing study could detect.

1. Question and setup

Question. A Vim user types text in insert mode and commands in normal mode: motions (`w b e j k`), operators, ⎋, `:` and `/`. What is the best layout for a programmer who uses Vim, what does keeping h j k l usable cost, where should Esc go, and how much does the answer depend on how much of the day is spent in Vim?

Board and cost. As in the programming study (`docs/papers/programming-layout/`): corne-36, letters on the base layer, Space and Backspace on base thumbs, and the measured layer-press cost (a held Shift, ×1.78; `layer_switch` 26.8). Enter is free here, so Enter and Esc compete for thumb keys and layer slots.

Text. `vim-programmer-typed` adds 22 % Vim sessions to the programming mix. The Vim part is synthetic (`gecko corpus vim`): a weighted grammar of normal-mode commands (motions ≈ 60 %, operators with motions and text objects, visual mode, `:w`, searches, leader maps) with short inserts typed from code, then ⎋. Its weights follow two published single-user logs; no population data on Vim keystrokes exists (`context/vim.md`). In the mix, ⎋ is 0.8 % of keystrokes, `:` 1.0 %, `/` 1.0 %, `j` 1.0 %, `k` 1.2 %. Two more mixes put the Vim share at 11 % and 33 % (`vimshare-11`, `vimshare-33`), and `programmer-typed` is 0 %.

h j k l policies (`context/vim.md`), each a hard constraint (`vim` weight 100):

- **free**: no constraint; the user remaps motions in their config.
- **pairs**: h left of `l` and `j` left of k, each pair on neighbouring keys; the pairs may sit apart.
- **row**: `h j k l` in order on four neighbouring keys of one row.
- **fingers**: four different fingers of one hand.
- **QWERTY spots**: `h j k l` pinned to their QWERTY keys.

Search. BLS, 6 seeds × 40 M evaluations per cell from a US-style layered design with Esc on the symbol layer; then 3 seeds × 160 M from each policy's best, because 40 M seeds spread 4–5 %, more than the policies differ. Each policy's optimum is scored **without** the h j k l term (`vim-free`), so the score difference is what the policy costs in typing.

2. The optimum

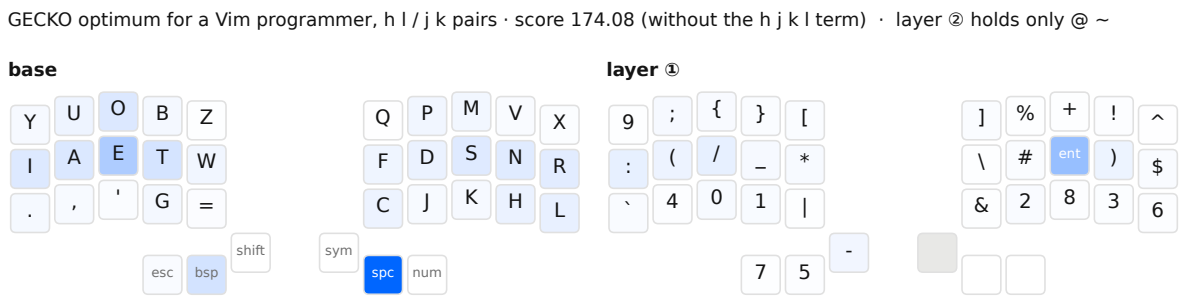


Figure 1: The recommended layout: h l and j k as neighbour pairs (`h l` and `j k` on the right bottom row), Esc on the left outer thumb, Enter on the symbol layer's right home row. Keys shaded by keystroke share in `vim-programmer-typed`. Layer ② holds only `@ ~`.

```
base                          layer L1 (held: right inner thumb)
y u o b z   q p m v x         9 ; { } [   ] % + ! ^
i a e t w   f d s n r         : ( / _ *   \ # ent ) $
. , ' g =   c j k h l         ` 4 0 1 |   & 2 8 3 6
 esc bsp sft  L1 spc L2          7 5 -
```

Run `20261025-061907-bls-108c69fa` (pairs policy, best of 3 × 160 M evaluations; 174.08 without the h j k l term).

- **Esc on a base thumb.** ⎋ is 0.8 % of keystrokes in the mix, 3.7 % of the Vim part, and every Vim session alternates it with letters. On a layer each Esc is a layer press (×1.78 under the measured cost); on a thumb it is free of that. Every policy's optimum and the 11 % and 33 % optima put it on the left outer thumb.
- **Enter moves to the layer.** Enter (2.6 % of keystrokes) leaves the thumbs for the symbol layer's home row, under the right middle finger. In code, Enter usually follows a symbol (`;`, `{`, `:`, `)`) that is already on the layer, and the model charges a layer only when it is entered, not when it is released, so Enter there is cheap; we did not measure that release cost. Without Vim (0 %), Enter returns to the thumb and Esc goes to a layer.
- **`:` and `/` on the layer's home row**, left pinky and middle, next to `(` and `_`. They are 1 % of keystrokes each; at 33 % Vim they move onto the base.
- **h j k l** sit as `j k` and `h l` on the right bottom row, `j` under the index. The letters around them are the programming optimum's kind: vowels `i a e` on the left home row, `s n r` on the right.

3. What h j k l policies cost

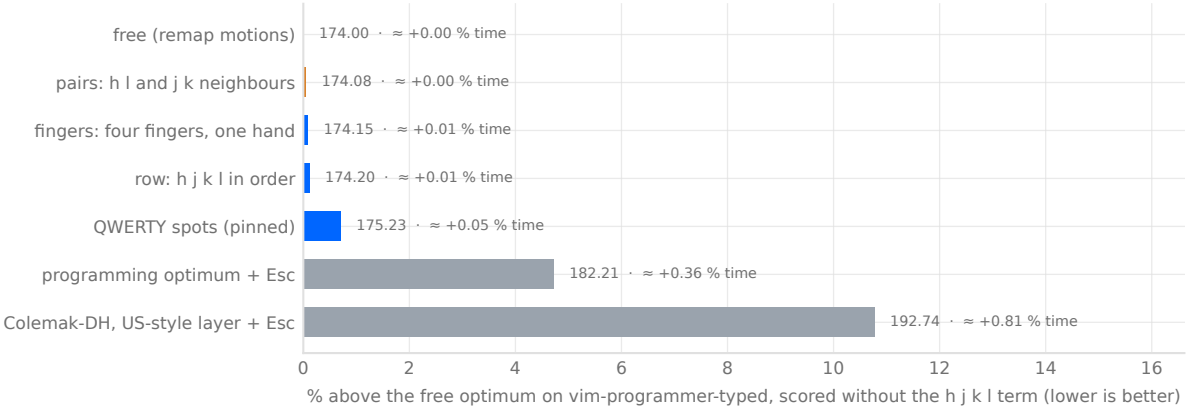


Figure 2: Each policy's optimum scored without the h j k l term on `vim-programmer-typed`, as % above the free optimum, with the gap in typing time per keystroke. Grey: the programming optimum with Esc added on a layer, and the US-style Colemak-DH start.

policy	score (40 M, best of 6)	score (160 M, best of 3)	vs free	≈ time
free	174.43	174.00	—	—
pairs	176.33	174.08	+0.05 %	+0.00 %
fingers	175.94	174.15	+0.09 %	+0.01 %

policy	score (40 M, best of 6)	score (160 M, best of 3)	vs free	≈ time
row	176.39	174.20	+0.11 %	+0.01 %
QWERTY spots	179.44	175.23	+0.71 %	+0.05 %
programming optimum + Esc (layer)		182.21	+4.7 %	+0.36 %
Colemak-DH, US-style layer + Esc		192.74	+10.8 %	+0.81 %

- **The shaped policies are free.** h j k l are mid-frequency letters, and on corne-36 the search has enough comparable keys to put a pair or a row where those letters would go anyway (here the right bottom row).
- **QWERTY spots cost 0.7 %:** four fixed keys on the right home row and index stretch that the search would give to other letters. That is the price of not relearning Vim at all, and it is small.
- **The 40 M results were noise.** At 40 M evaluations seeds spread 3–5 % and the policies seemed 1–3 % apart; at 160 M the three seeds of each policy agree within 0.2–0.9 % and the policies within 0.1 %. Earlier ansi-30 results (`context/vim.md`: +2 % to +24 %, SA 60 s) may carry the same noise, and a 30-key board has fewer keys to absorb the constraint; we did not re-examine them.

4. How much Vim?

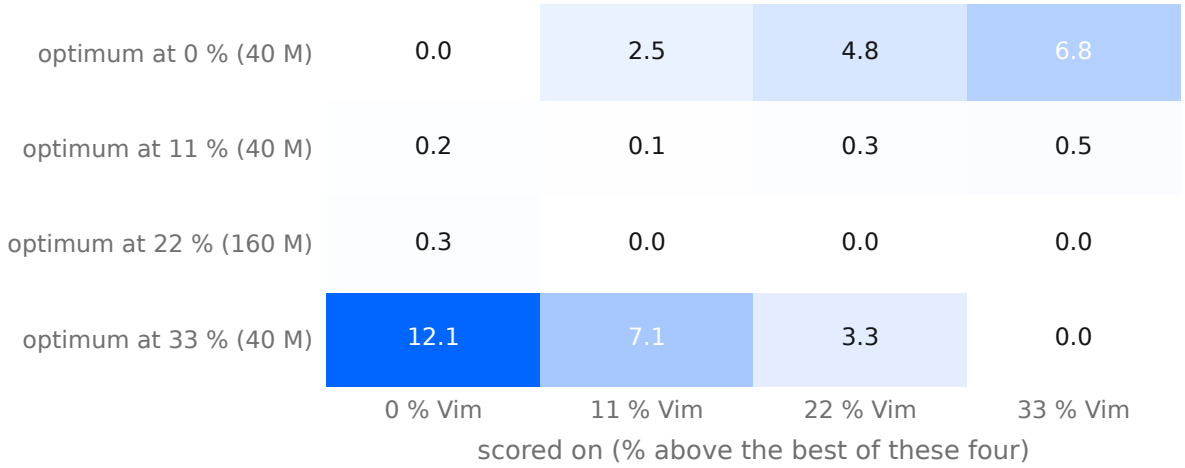


Figure 3: The optima for 0, 11, 22 and 33 % Vim (rows) scored on each mix (columns), % above the best of the four. The 22 % optimum (160 M) is within 0.3 % everywhere; the 33 % optimum (40 M) puts `:`, `/` and Enter on the base and loses 7–12 % on less Vim.

- **One layout for 11–33 % Vim.** The 22 % optimum is the best found on the 11 %, 22 % and 33 % mixes and 0.3 % behind the 0 % optimum on programming without Vim. The 11 % and 33 % optima came from 40 M searches, so part of their gap is search noise.
- **Vim or not changes Esc, not the letters.** Without Vim the optimum puts Esc on a layer and Enter on the thumb, and that layout is 4.8 % worse on the Vim mix.
- The programming optimum (`docs/papers/programming-layout/`) with Esc added on a layer is 4.7 % worse on the Vim mix; the Vim optimum is 0.4 % worse than it on programming without Vim. **A programmer who uses Vim at all should take the Vim layout.**

5. How sure is it?

Search. At 40 M evaluations BLS is not reliable on this problem (seeds spread 3–5 %); every claim above uses the 160 M searches (3 seeds per policy, started from that policy's best 40 M layout), whose seeds agree within 0.2–0.9 %. They are not exact optima.

Text. The Vim keystrokes come from a grammar, not from logs: motion ≈ 60 %, inserts of about 12 characters, `:w`, searches. No study reports what a population of Vim users types (a single-user analyzer sample has insert mode 44 % and normal mode 42 % of keys). Esc and Enter placement follow from those frequencies; replace the `vim` component with your own `nvim -w` log (`gecko corpus vim --log`) before trusting the details.

Cost. The layer cost is a held Shift (programming study), and layer releases are free; Enter on the layer depends on that. The gaps between policies are below 0.1 % of typing time: they say the policies cost nothing measurable, not that one is better.

6. Conclusions

1. **Keep h j k l in any shape you like.** Pairs, row or fingers cost $\leq$ 0.1 %, and QWERTY spots 0.7 %; pick by habit, not by score.
2. **Put Esc on a base thumb**, and let Enter move to the symbol layer's home row next to `;` and `)`.

3. **`:` and `/` belong on the symbol layer's home row** unless Vim is a third of your typing.
4. **One layout covers 11–33 % Vim**, and costs 0.3–0.4 % if you stop using Vim; the programming layout with Esc on a layer costs 4.7 % if you start.

7. Reproduce

```
cd bench/vim-layout-2026-10-25
./grid.sh && ./grid2.sh       # policies and shares at 40 M, then 3 × 160 M per policy
python3 -I analyze.py         # → results.json
MPLCONFIGDIR=/tmp/mpl python3 -I ../../docs/papers/vim-layout/figures.py
python3 -I ../../docs/papers/vim-layout/build.py
gecko verify run:20261025-061907-bls-108c69fa
```